

v6 SSD Protocol Oracle

Technical Report

Deterministic Rule-Based TCG Storage Compliance Verifier

Project	DL2026 SSD Protocol Oracle
Version	v6
Author	eric-mjk
Date	2026-06-06
Language / Runtime	Python 3 - fully offline, no network or ML
Public Dataset Score	100.00 % (20/20)
Synthetic Suite Score	100 % (84/84)

This report describes the design, internal architecture, and specification coverage of the v6 SSD protocol oracle. The oracle deterministically judges whether the final response in a TCG Storage / Opal command-response trajectory is compliant with the TCG Core and Opal SSC v2 specifications. It requires no machine-learning model, no internet access, and no GPU; every verdict is produced by an explicit rule derived from the published specifications.

1. Project Overview

The DL2026 SSD Protocol Oracle project builds a verifier for TCG Storage (Opal SSC v2) command-response trajectories. A trajectory is a chronological list of host-issued commands and device responses recorded from a real or simulated SSD. The oracle reads the entire trajectory and produces a single binary verdict for the final response: pass if the final response is protocol-compliant with the accumulated state, fail otherwise.

Key clarification: 'pass' does NOT mean the SSD returned SUCCESS. 'pass' means the final response is consistent with what the TCG/Opal specification mandates given the prior events. An error response can be 'pass' if the specification requires an error in that context.

1.1 Problem Statement

Given a full command-response trajectory (JSON list of steps), determine whether the device response on the last step conforms to the TCG Core and Opal SSC v2 specifications. The verifier must operate offline and produce deterministic, reproducible verdicts.

1.2 Scope - TCG / Opal Specifications Covered

- * TCG Core Specification (session management, ACE/AccessControl, method lifecycle, C_PIN, clock, crypto streams, log tables)
- * TCG Opal SSC v2 (AdminSP, LockingSP, locking ranges, MBR shadow, MSID/SID/PSID authorities, SP lifecycle, re-encryption)
- * Level 0 Discovery (IF_RECV feature descriptor validation)
- * Media-encryption key management (GenKey, re-encryption state machine)

1.3 Non-Goals

- * Does not test storage performance or data integrity.
- * Does not require a live SSD; works entirely on recorded trajectories.
- * Does not use a machine-learning model or heuristics.

2. Architecture

2.1 High-Level Data Flow

The solver pipeline is strictly sequential and split across five Python modules. Each module has a single responsibility, making the code auditable against the specification section-by-section.

```
raw JSON trajectory (list of steps)
  v normalize_trajectory()          normalizer.py
canonical event list
  v track_state(events[:-1])        state.py
protocol state dict
  v judge_final(state, events[-1])  oracle.py
RuleResult -> verdict ("pass" | "fail")
```

2.2 Module Breakdown

Module	File	Responsibility
Entry Point	<code>solver.py</code>	<code>Solver.predict</code> / <code>predict_one</code> ; debug summary
Normalizer	<code>normalizer.py</code>	Raw JSON -> canonical event dict
State	<code>state.py</code>	Prefix replay; <code>ProtocolState</code> mutations
Oracle	<code>oracle.py</code>	Final-event judging; method dispatch; ACE evaluation
Spec Docs	<code>spec_docs.py</code>	Spec metadata, column maps, rule refs, coverage
Spec Tables	<code>spec_tables.py</code>	Legacy static Opal policy constants

2.3 Entry Point - `solver.py`

`Solver.predict(dataset)` iterates over the dataset list (each item has 'id' and 'steps') and returns a dict mapping id -> verdict. `Solver.predict_one(steps)` handles a single trajectory. If `SOLVER_DEBUG=1` is set, each verdict is accompanied by a structured debug line showing the final event kind, status, expected status, spec refs, and the full state summary.

```
events = normalize_trajectory(steps)  # list of canonical event dicts
state  = track_state(events[:-1])     # replay all but the last event
result = judge_final(state, events[-1]) # oracle judgment on final event
return result.verdict                 # 'pass' or 'fail'
```

3. Normalizer (normalizer.py)

The normalizer converts raw JSON records into a uniform canonical event dict. This decouples the rest of the pipeline from JSON formatting quirks and naming inconsistencies in the test dataset.

3.1 Event Kinds

- * method - a TCG method invocation (StartSession, Get, Set, Authenticate, etc.)
- * read - a host data-read command (LBA range access)
- * write - a host data-write command (LBA range access)
- * discovery - an IF_RECV Level 0 Discovery response
- * command - any other host command (reset, power cycle, etc.)

3.2 Key Normalizations

Field	Input variants	Canonical form
status	SUCCESS / pass / PASS	success
status	NOT_AUTHORIZED / notauthorized	not_authorized
status	INVALID_PARAMETER / invalidparam	invalid_parameter
authority	0x0000000900000006 (UID)	SID
authority	0x0000000900010001 (UID)	Admin1
sp	0x0000020500000001 (UID)	AdminSP
sp	0x0000020500000002 (UID)	LockingSP
object	0x0000000B00008402 (UID)	C_PIN_MSID
object	0x0000080200000001 (UID)	Locking_Global

3.3 Column and Parameter Extraction

Values and Cellblock parameters are parsed into integer-keyed column dicts. Column names (e.g. 'RangeStart') are resolved to their spec-defined column numbers via `spec_docs.column_number_for_name()`. This handles both numeric keys (3, 0x03) and named keys, ensuring the state and oracle layers always operate on consistent integer column indices.

3.4 Level 0 Discovery Normalization

An IF_RECV command whose output contains a 'discovery' or 'descriptors' key is converted into a kind='discovery' event. Each feature descriptor is stored in a dict keyed by its integer feature code (e.g. 0x0001 = TPer Feature, 0x0002 = Locking Feature, 0x0203 = Opal SSC v2).

4. State Tracker (state.py)

track_state(events) processes the prefix (all events except the last) in order, applying each successful event to the protocol state. The fundamental invariant is: only successful operations mutate state.

4.1 Protocol State Fields

Field	Type	Description
session	dict	open, sp, write, authorities, had_failure, trusted
credentials	dict	SID, MSID, Admin1..4, User1..8 PIN values
sp_lifecycle	dict	AdminSP / LockingSP lifecycle state
locking_sp_active	bool	True after successful LockingSP Activate
locking_ranges	dict	Range name -> config (bounds, lock flags, reencrypt)
mbr	dict	MBRControl columns (enable, done, done_on_reset)
ace_rows	dict	ACE UID / name -> BooleanExpr + columns policy
access_control_rows	list	AccessControl rows linking objects to ACE refs
authority_rows	dict	Authority name -> enabled, class, operation, secure
key_generations	dict	Object/range -> GenKey invocation count
trylimit_by_authority	dict	Authority -> C_PIN.TryLimit value
failed_auth_counts	dict	Authority -> consecutive failed attempts
writes	dict	LBA tuple -> write record with key-gen snapshot
log_tables	set	Known log table UIDs

4.2 Per-Method State Mutations

- * StartSession: opens session, sets sp/authority/write, resets TryLimit counter on success.
- * Authenticate: adds authority to session set; on failure increments failed_auth_counts.
- * EndSession / CloseSession: clears session to empty.
- * Get: captures return columns into tables dict; updates credentials / locking ranges / MBRControl as appropriate.
- * Set: updates credentials, locking range columns, MBRControl, ACE rows, AccessControl rows, authority rows.
- * Activate(LockingSP): sets locking_sp_active=True; copies SID credential to Admin1 on first activation only.
- * GenKey: increments key_generations counter; invalidates C_PIN credential if target is C_PIN.
- * Revert(LockingSP): resets locking ranges, MBR, LockingSP credentials to factory defaults; bumps key generations for all ranges.
- * Revert(AdminSP): additionally resets AdminSP credentials and all table data.
- * RevertSP: same as Revert but invoked inside the target SP session; supports KeepGlobalRangeKey flag.
- * Power-cycle / reset command: applies LockOnReset to all locking ranges; resets MBR.Done if DoneOnReset fires; clears TryLimit counters; closes session.

4.3 TryLimit / Authority Lockout

When a C_PIN TryLimit (column 5) is observed via Get or Set, it is stored in trylimit_by_authority. Every failed Authenticate or failed authenticated StartSession increments failed_auth_counts for that authority. When failed_auth_counts >= TryLimit the oracle will expect authority_locked_out status on subsequent

auth attempts.

4.4 Re-encryption State Machine

Locking range column 12 (ReEncryptState) follows a five-value state machine defined in TCG Core spec section 5.7.3. The tracker applies ReEncryptRequest (column 13) transitions according to the VALID_REENCRYPT_REQUEST_STATES matrix and updates ReEncryptState, ActiveKey, and NextKey accordingly. RangeStart/RangeLength writes are blocked while the range is not IDLE.

5. Oracle (oracle.py)

`judge_final(state, event)` dispatches on event kind and method name to a dedicated judge function. Each judge function returns a `RuleResult` with a verdict, confidence, reason, expected/actual status class, spec refs, and policy source.

5.1 Status Class Model

Rather than comparing raw status strings, the oracle classifies statuses into equivalence classes. This allows a single rule to accept any auth-error variant without enumerating all possible strings.

Class	Statuses included	Meaning
success	success	Operation completed normally
auth_error	not_authorized, authority_locked_out	Authentication/authorization failure
invalid_parameter	invalid_parameter, invalid_command, ...	Malformed request
resource_error	sp_busy, sp_disabled, sp_frozen, ...	Resource or lifecycle constraint
error	fail, None	Generic or unknown failure

5.2 Method Dispatch Table (selected methods)

Method	Judge Function	Key checks
StartSession	<code>judge_start_session</code>	SP lifecycle, authority enabled/lockout, credential match
Authenticate	<code>judge_authenticate</code>	Authority type, enabled, lockout, credential match
Get	<code>judge_get</code>	Cellblock validity, (N) columns, ACE policy, family rules
Set	<code>judge_set</code>	Column validity, read-only cols, locking range geometry
Activate	<code>judge_activate</code>	Target=LockingSP, AdminSP write session with SID
Revert	<code>judge_revert</code>	SP family target, AdminSP write session with SID or PSID
RevertSP	<code>judge_revert_sp</code>	LockingSP active, write session, KeepGlobalRangeKey checked
GenKey	<code>judge_gen_key</code>	Media-key target, range re-encrypt idle, ACE policy
Random	<code>judge_random</code>	Count <= 32, open session
Read (data)	<code>judge_read</code>	Lock state, key-generation consistency
Write (data)	pass-through	Write records tracked; outcome judged on prefix
IF_RECV	<code>judge_discovery</code>	Required feature descriptors, LockingEnabled vs state

5.3 ACE / AccessControl Policy Engine

The oracle evaluates dynamic ACE/AccessControl policy loaded from `spec_docs.py` (pre-seeded from the TCG Opal spec) and updated by any `Set/AddACE/RemoveACE` operations seen in the prefix. The evaluation pipeline is:

- * Find AccessControl rows whose invoking UID and method match the event.
- * For each row, resolve ACE references from `ace_rows`.
- * Evaluate the ACE BooleanExpression against the set of session authority tokens (authority + its class memberships, e.g. Admin1 -> {Admin1, Admins}).
- * If expression evaluates True and requested columns are in the ACE's allowed-column set: allow.
- * If expression evaluates False and no unknown rows remain: deny.
- * If no determination can be made: return None (fall through to family-level rules).

5.4 Preflight Checks (method_preflight)

Before any method-specific logic, `method_preflight()` validates structural correctness of the request. These checks are independent of state:

- * Method name is in the supported method list (METHOD_NAMES).
- * Required parameters are present (SPID for StartSession, etc.).
- * Integer parameters are non-negative (HostSessionID, Count, etc.).
- * Boolean parameters are parseable (Write, KeepGlobalRangeKey).
- * Set Values column indices are within spec-defined range for the target family.
- * Cellblock start $\leq$ end, within the max column for the target family.
- * Invalid re-encryption request / next-key update per state machine.
- * Session is open when required; write session when required.

5.5 Locking Range Access (Data Reads / Writes)

Data Read and Write commands are judged by looking up the locking range that covers the requested LBA.

The `lock_state_for_lba()` function returns whether locking is enabled and whether the range is currently locked. A read to a read-locked range should fail; a write to a write-locked range should fail.

Key-generation consistency is also verified: if the media key for a range was rotated (GenKey) after a write, reading back the old data must fail (data is cryptographically erased).

6. Specification Coverage

Coverage is tracked at the rule level. Each oracle rule carries a `spec_refs` tuple pointing to the precise specification section that justifies the rule. The `spec_docs.py` module maintains a `RULE_REFERENCES` registry and a coverage report is available in `artifacts/spec_coverage_report.json`.

6.1 Coverage Summary by Domain

Domain	Key spec sections	Status
Session management	core/5.2, core/5.3.4	Implemented
Authentication	core/5.3.4.1, opal/4.2.1.7	Implemented
C_PIN credential	opal/4.2.6.1, core/3.3.7.4	Implemented
Authority lifecycle	opal/4.3.1.8, core/5.3.4.1.3	Implemented
TryLimit / lockout	core/3.3.7.4, core/5.3.4.1.1	Implemented
Locking ranges	opal/4.3.1, core/5.7.2	Implemented
MBRControl / MBR	opal/4.3.1.6, opal/3.2.3	Implemented
SP lifecycle (Activate/Revert)	opal/5.1	Implemented
Re-encryption	core/5.7.3	Implemented
ACE/AccessControl	core/5.3.4.3	Implemented
Level 0 Discovery	opal/3.1.1	Implemented
Media key (GenKey)	core/5.3.4.1.1.1	Implemented
Clock methods	core clock template	Implemented
Crypto streams	core crypto template	Implemented
Log tables	core/5.8.3	Implemented
PSID Revert	Opal PSID Feature Set	Implemented
Meta-ACL (AddACE etc)	core meta-ACL	Implemented

6.2 Notable Edge Cases

- * Repeated Activate on an already-Manufactured LockingSP is a no-op (opal/5.1.1); SID is NOT re-copied to Admin1 on the second call.
- * Disabled authorities must return SUCCESS with result=False on Authenticate (not NOT_AUTHORIZED) per core/5.3.4.1.
- * Key-exchange authorities (Exchange, TPerExchange, TPerSign) also return SUCCESS with result=False on Authenticate (core/5.3.4.1.3).
- * C_PIN PIN column (col 3) is write-only (NOPIN) except for C_PIN_MSID via ACE_C_PIN_MSID_Get_PIN (opal/4.2.6.1).
- * AccessControl (N) columns (1,2,4,8) cannot be read by Get regardless of ACE policy.
- * KeepGlobalRangeKey=True in RevertSP fails if Global Range is both read-locked and write-locked (opal/5.1.3.2).
- * DoneOnReset is triggered on power cycle - MBR.Done resets to False (opal/4.3.5.3, opal/3.2.3).
- * SyncSession response to a successful StartSession must include non-zero HostSessionID and SPSessionID (core/5.2.3.2).

7. Testing and Evaluation

7.1 Public Dataset

The grader evaluates the solver on the public dataset (dataset/testcases/, labels in dataset/label.jsonl) using evaluate.py. The solver achieves 100.00% accuracy (20/20 test cases).

```
cd v6
python3 evaluate.py
```

7.2 Synthetic Regression Suite

The v6/customtest_84/ directory contains 84 hand-crafted synthetic test cases that exercise specific rules and edge cases not covered by the public dataset. The suite is generated and checked via generate_synthetic.py.

```
cd v6/customtest_84
python3 generate_synthetic.py --check-only
```

All 84 synthetic cases pass (100%).

7.3 Debug Mode

Setting SOLVER_DEBUG=1 enables per-event debug output. The debug line includes: final event kind and method, object UID and family, observed status, expected status class, actual status class, verdict, policy source, coverage status, spec refs, and a full state snapshot.

```
SOLVER_DEBUG=1 python3 v6/evaluate.py
```

7.4 Static Checks

```
# Compile-time syntax check (no runtime deps needed)
python3 -m py_compile v6/src/*.py
```

8. Design Decisions

8.1 Why Deterministic Rules (not ML)?

The TCG/Opal specifications are precise and complete for the operations being verified. A rule-based approach is preferred because it is auditable (each verdict links to a spec section), reproducible (same trajectory always gives same verdict), and requires no training data or GPU.

8.2 Status-Class Comparison

The oracle does not compare raw status strings. Instead it uses a five-class equivalence model (success, auth_error, invalid_parameter, resource_error, error). This is necessary because the specification defines expected behaviour in terms of status classes, not specific codes. For example, 'the TPer SHALL return an error' encompasses fail, sp_failed, tper_malfunction, etc.

8.3 Only-Successful-Operations Mutate State

A failed Set does not change credentials. A failed Revert does not reset ranges. This is a first-class design invariant, not a special case. The apply_event() function checks success_like(event) before dispatching to any mutation handler.

8.4 Five-Module Architecture

Splitting v5's monolithic solver.py into five modules (normalizer, state, oracle, spec_docs, solver) aligns the code structure with the verification pipeline stages. Each module can be read and audited independently against the corresponding spec sections. It also allows the normalizer to be tested without a full state/oracle round-trip.

8.5 Confidence Field in RuleResult

RuleResult carries a confidence float (0.0-1.0) alongside the verdict. A confidence < 1.0 signals that the oracle could not fully determine the expected outcome from the tracked state (e.g. no tracked credential for an authority). In these cases, the oracle passes through whichever status the device returned, as long as it is in the plausible class (success or auth_error for an unknown credential situation).

9. Submission Layout

The grader expects the following directory structure inside the v6/ folder:

```
v6/  
  src/  
    solver.py          # Solver class - grader entry point  
    normalizer.py      # Raw JSON -> canonical events  
    state.py           # Protocol state tracker  
    oracle.py           # Final-event judge  
    spec_docs.py       # Spec metadata and rule refs  
    spec_tables.py     # Legacy static policy constants  
  artifacts/  
    spec_index.json    # Spec-derived ACE/AccessControl seed data  
    spec_coverage_report.json  
  setup.sh             # Environment setup  
  pyproject.toml       # Project metadata and dependencies  
  uv.lock              # Locked dependency tree
```

The grader instantiates `Solver()` and calls `predict(dataset)` where `dataset` is a list of `{id, steps}` dicts. The solver returns a dict mapping each id to 'pass' or 'fail'. No network access is required at evaluation time.